

UNDERSTANDING DEEP LEARNING

지도학습과 얇은 신경망

Supervised Learning & Shallow Neural Networks

Chapter 2 & 3 · 조은빈

강의 개요 Agenda

선형회귀에서 출발해 얇은 신경망까지

PART 1 · Chapter 2

지도학습 Supervised Learning

- 모델 · 추론 · 파라미터의 의미
- 1D 선형회귀
- Loss(손실)와 손실 표면
- 경사 하강법
- 일반화와 과소·과대적합

PART 2 · Chapter 3

얇은 신경망 Shallow Networks

- ReLU와 선형 함수
- 세 직선 → 함수 만듦
- 다중 입력·출력과 2D 처리
- 표현력: linear region의 증가
- 용어 정리 · 역전파 · 왜 ReLU를 사용?

01

지도학습

Supervised Learning

CHAPTER 2

지도학습이란? Overview

입력 x 를 받아 출력 y 를 예측하는 함수를 데이터로부터 찾는 일.



추론 Inference

입력을 함수에 넣어 출력을 계산하는 과정
이때, 이 함수를 모델이라 부름

$$y = f[x, \phi]$$

파라미터 Parameters ϕ, θ

모델식에는 파라미터 ϕ, θ 가 들어 있음. 이 값에 따라 입력-출력 관계가 달라짐. 이때, 모델은 가능한 관계들의 **가족(family)**을 나타냄.
(다음 슬라이드 예시)

학습 Training

loss를 최소화하는 파라미터를 찾는 과정
예) 중고차의 나이·주행거리 → 예상 가격.

핵심

학습(training)의 목표는 결국

“Loss를 가장 작게 만드는 파라미터를 찾는 것”

1D 선형회귀 1D Linear Regression

가장 단순한 모델 — 입력과 출력의 관계를 직선 하나로 나타냄

$$y = f[x, \boldsymbol{\phi}] = \phi_0 + \phi_1 x$$

필기

1D : 입력이 하나(x)인 경우

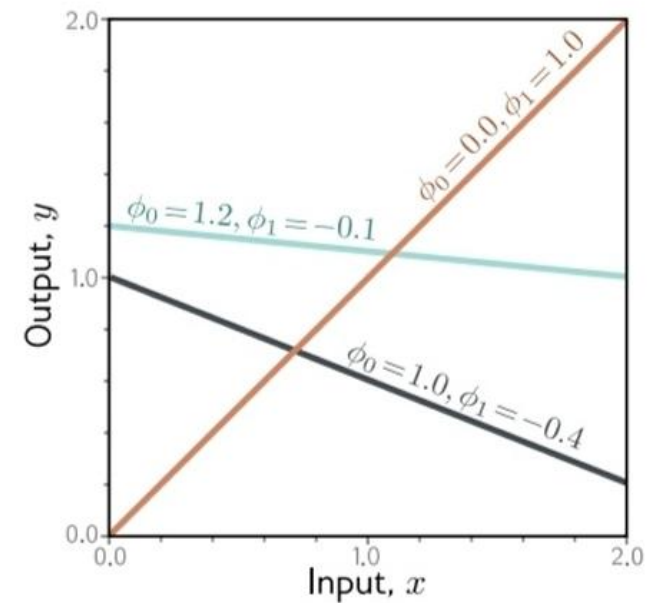
필기

파라미터를 바꾸면 **직선이 바뀐다** (오른쪽 그래프 참고)

필기

선형회귀는 직선 하나가 아니라 **가능한 모든 직선**을 표현하는 모델이며, 그중 **loss가 최소**인 직선(최소제곱회귀직선)을 찾아야 한다.

$y = \phi_0 + \phi_1 x$ 의 가족



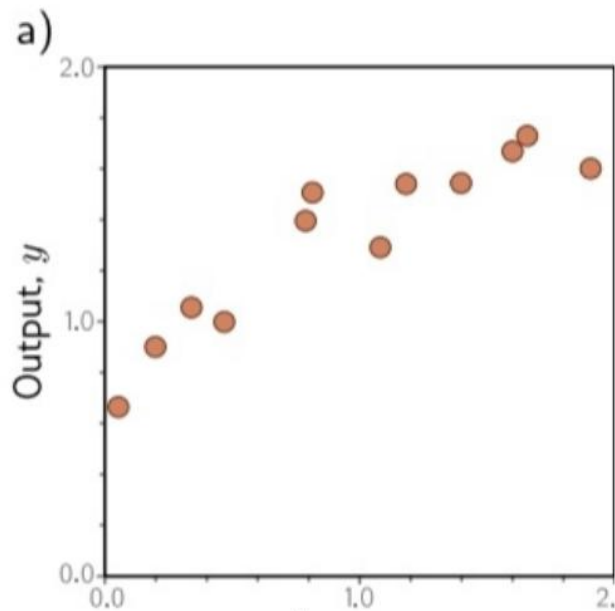
파라미터(절편 ϕ_0 , 기울기 ϕ_1)에 따라 달라지는 직선들

Figure 2.1

손실 함수 Loss

모델이 얼마나 틀렸는지를 하나의 숫자로 나타냄

학습 데이터



입력·출력 쌍의 집합

Figure 2.2a

$$L[\phi] = \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

필기

Loss : 얼마나 틀렸는지(오차)를 숫자로 나타낸 값.
각 점에서 **예측 $f(x_i)$** 와 **실제 y_i** 의 차이를 모음.

왜 제공?

제공하는 이유

- ① 오차의 부호를 없애기 위해
- ② 큰 오차를 더 크게 벌주기 위해.

이렇게 만든 오차제곱합을 **least-squares loss**라 부름.

어떤 직선이 더 좋은가 Comparing Fits

오차제곱합이 작을수록 데이터를 더 잘 설명하는 모델

Loss = 7.07

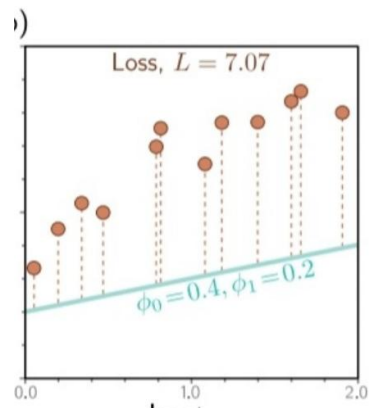


Figure 2.2b

Loss = 10.28

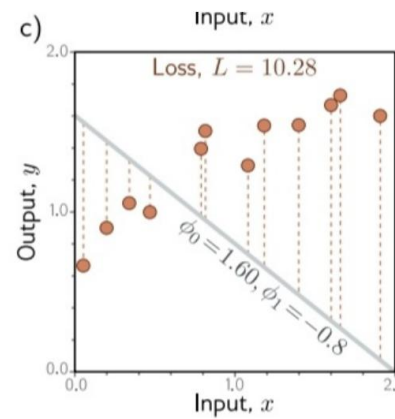


Figure 2.2c

Loss = 0.20 [최적]

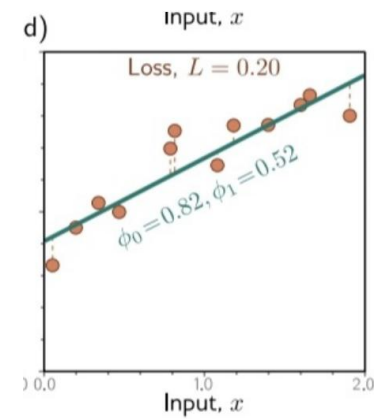


Figure 2.2d

필기

점선 = 점과 직선 사이 거리 = 오차(loss).

필기

loss가 작을수록 좋으므로 (d)가 가장 좋은 모델.

손실 표면 Loss Surface

모든 파라미터 조합의 loss를 그리면 하나의 표면이 됨

3D 손실 표면

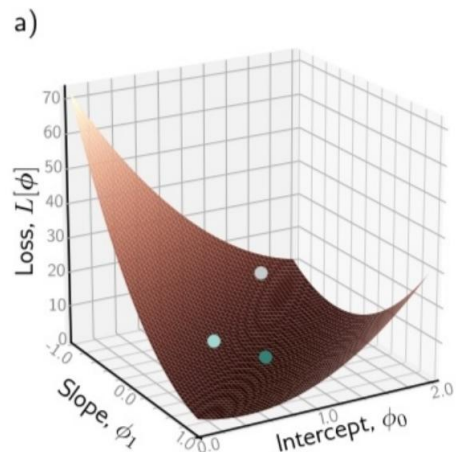


Figure 2.3a

위에서 본 모습

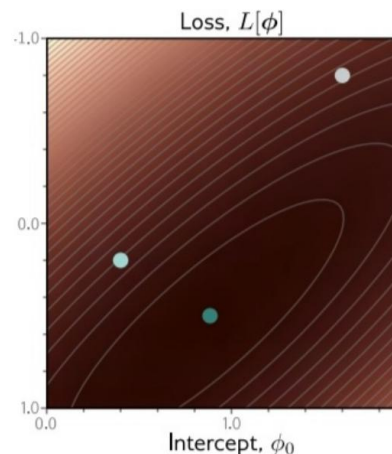


Figure 2.3b

직관

난해해 보여도 **짚지 마삼**. 그냥 모든 직선(다 모으면 평면)에 대한 loss를 **3차원**으로 그린 것!

필기

(b)는 (a)를 **천장에서** 본 것 — 초록 점이 최소 loss를 만드는 최적의 파라미터값.
이 파라미터로 가장 좋은 직선을 만들 수 있다.

필기

가장 **움푹한** 곳이 loss가 최소인 최적 파라미터 $\hat{\phi}$
회색 선은 등고선 — loss가 같은 점들을 이은 선.

$$\hat{\phi} = \operatorname{argmin}_{\phi} \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

$$\hat{\phi} = \operatorname{argmin} [L(\phi)]$$

학습: 경사 하강법 Gradient Descent

표면 위를 가장 가파른 내리막으로 조금씩 내려가 최소인 loss를 찾음.

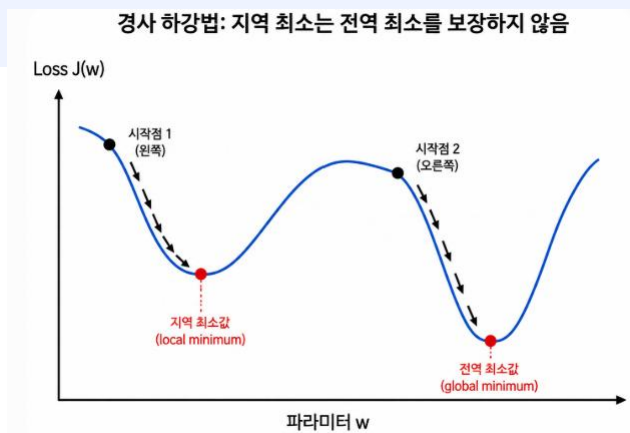
필기

경사 하강법 : 랜덤한 파라미터에서 시작해 loss 함수 위 가장 가파른 내려가는 방향으로 조금씩 반복 이동하여 최소 loss를 찾는 방법.

한계 → 지역 최소가 전역 최소를 보장하지 못함.

필기

Figure 2.3(b)의 최적화 과정을 **0→4**로 표시. b)는 그 0~4가 실제 어떤 직선인지 보여줍니다.



0 → 4 로 내려가기

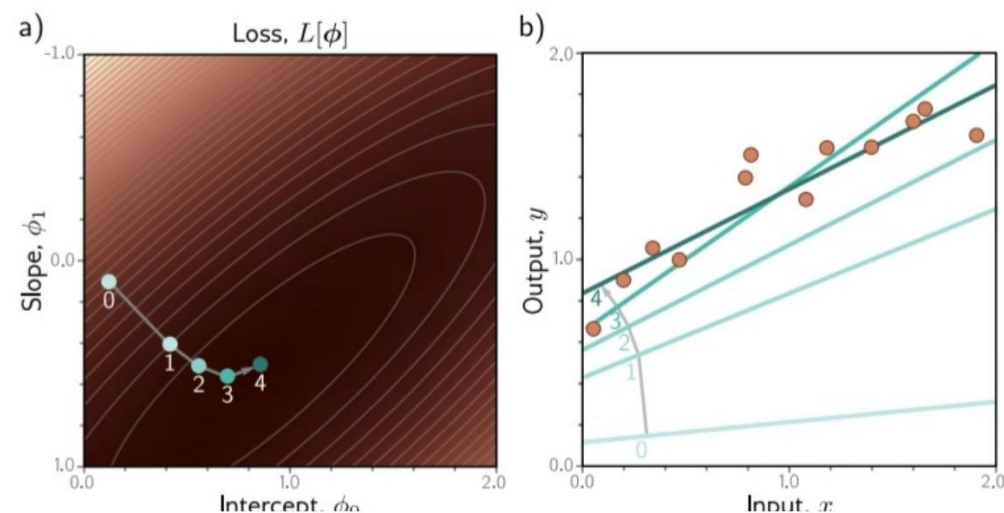


Figure 2.4 · 경로(좌) & 해당 직선(우)

일반화와 과적합 Generalization & Overfitting

학습이 끝나면 새로운 데이터(test data)에서의 성능을 확인해야 합니다.

과소적합 · Underfitting

모델이 너무 **단순**해 관계를 못 잡음 →
testing loss 큼

적절 · Good fit

복잡도가 **알맞아** 새 데이터에도 잘 맞춤
→ **testing loss 작음**

과대적합 · Overfitting

너무 **복잡**해 학습 데이터의 지엽까지 외
움 → **testing loss 큼**

정의

일반화(Generalization) = 새로운 데이터에서도 잘 맞는 능력.
과소·과대적합 모두 testing loss가 커집니다.

비유

training = 기출, **testing** = 수능.
기출만 달달 외우면(과대적합) 수능에서 무너집니다.

보충 개념 Notes

헛갈리기 쉬운 두 쌍의 용어 정리

Loss vs Cost

loss = 데이터 한 개의 오차

cost = 전체의 오차(전체 loss의 합).

※ But, 실제로는 둘을 별로 구분하지 않고 섞어 씁니다.

Discriminative vs Generative

Discriminative : 입력 $\rightarrow$ 출력 $y = f(x, \varphi)$

(예: 나이(x) $\rightarrow$ 연봉(y))

Generative : 입력 $\leftarrow$ 출력 $x = g(y, \varphi)$

(예: 나이(x) $\leftarrow$ 연봉(y))

필기

데이터가 많아지면서, 사전지식을 넣은 generative보다 복잡한 **Discriminative Model**이 더 좋은 성능을 내게 됨.
그래서 현대 머신러닝은 대부분 discriminative 방식입니다.

보충 개념 Notes

Loss vs Cost

Loss

MSE $L_i = (y_i - \hat{y}_i)^2$

RMSE $L_i = (y_i - \hat{y}_i)^2$

MAE $L_i = |y_i - \hat{y}_i|$

Cross entropy $\hat{p}_{ik} = \frac{e^{z_{ik}}}{\sum_{j=1}^C e^{z_{ij}}} \quad L_i = -\log(\hat{p}_{ic})$

Cost

MSE $J(\theta) = \sum_{i=1}^N (y_i - \hat{y}_i)^2$

RMSE $\text{RMSE} = \sqrt{\text{MSE}}$

MAE $J(\theta) = \sum_{i=1}^N |y_i - \hat{y}_i|$

Cross entropy $J(\theta) = -\sum_{i=1}^N \sum_{k=1}^C y_{ik} \log(\hat{p}_{ik})$

필기

책에서는 cost를 loss의 합이라 정의했지만, 실제 딥러닝에선 loss의 평균(합을 N으로 나눠줌)을 cost라 간주할 때도 많다.

보충 개념 Notes

Discriminative vs Generative

Discriminative

입력 $\rightarrow$ 출력 $y = f(x, \varphi)$

관측된 데이터 x 를 모델에 넣어 **정답** y 를 예측

(예: 강아지 사진 $x \rightarrow$ 모델 $f \rightarrow$ 강아지 y)

Generative

입력 $\leftarrow$ 출력 $x = g(y, \varphi)$

예) 만약 고양이라면, 이런 사진 x 가 나올 확률이 얼마나 높을까?

만약 강아지라면, 이런 사진 x 가 나올 확률이 얼마나 높을까?

$p(x | y = \text{고양이})$

$p(x | y = \text{강아지})$

둘 중 더 확률이 높은 쪽을 고름.

필기

데이터가 많아지면서, 사전지식을 넣은 generative보다 **복잡한 Discriminative Model**이 더 좋은 성능을 내게 됨.

그래서 현대 머신러닝은 대부분 discriminative 방식입니다.

2장 요약 Chapter 2 Summary

1

모델은 함수 $y=f[x,\varphi]$

2

Loss로 오차를 수치화

3

경사 하강법으로 loss최소화

4

test로 모델의 일반화 확인

- 모델·추론: 파라미터 φ 가 입출력 관계의 가족(family)을 정의하고, 함수에 입력을 넣어 출력을 계산한다.
- 학습: 오차제곱합(least-squares loss)이 최소가 되는 $\hat{\varphi}$ 를 경사 하강법으로 찾는다.
- 한계와 평가: 지역 최소 문제가 있고, 과소·과대적합을 피해 일반화 성능을 확인해야 한다.
- 다음 장: 직선 하나로는 부족 → 더 풍부한 함수를 만드는 얇은 신경망으로.

02

얕은 신경망

Shallow Neural Networks

CHAPTER 3

용어 정리 Terminology

신경망을 이야기할 때 쓰는 표준 용어

층 · 뉴런 · 가중치

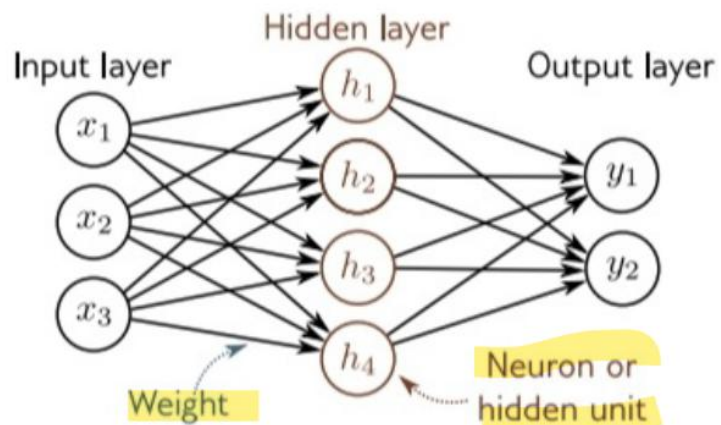


Figure 3.12

핵심 용어

- Node 노드 — 입력·은닉·출력의 모든 점(왼쪽 그림에서의 원 하나)
- Neuron 뉴런 — 은닉층, 출력층 계산 단위(output neuron, hidden unit=hidden neuron)
- parameter 파라미터 — Weight 가중치 + bias 편향 (상수)
- Weight 가중치 — $w = \theta_{11}$ (기울기)
- Pre-activation — $\theta_{10} + \theta_{11}x$ 계산값 (예: -3)
- Feed-forward — 입력→은닉→출력 한 방향(순전파=계산)
- Fully connected — 모든 노드가 다음 층 모든 노드와 연결

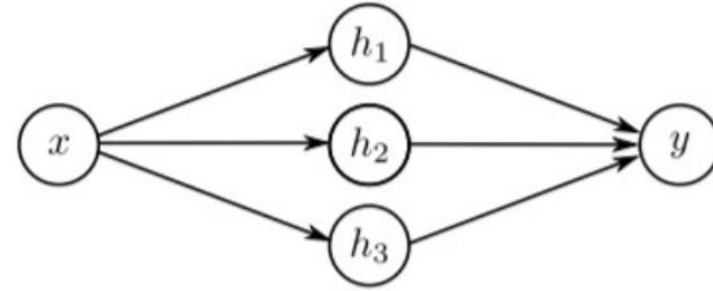
얇은 신경망이란? Shallow Neural Network

직선 하나로는 부족하므로 은닉층 하나를 넣어 만든 더 풍부한 함수

입력층 Input → 은닉층 Hidden → 출력층 Output

기존 1D 선형회귀 모델 (은닉층 X)

$$y = f[x, \phi] = \phi_0 + \phi_1 x$$



예시) 10개의 파라미터

$$y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

스칼라 입력 x 를 받아 상수와 기울기에 파라미터(θ)를 설정하여 은닉층 뉴런 개수만큼 직선을 만든다. (여기선 3개)
세 직선을 활성화 함수 $a[\cdot]$ 에 통과시키고(보통 ReLU 사용),
그 결과를 ϕ 로 가중합한다. 이를 통해 스칼라 출력 y 를 구한다.

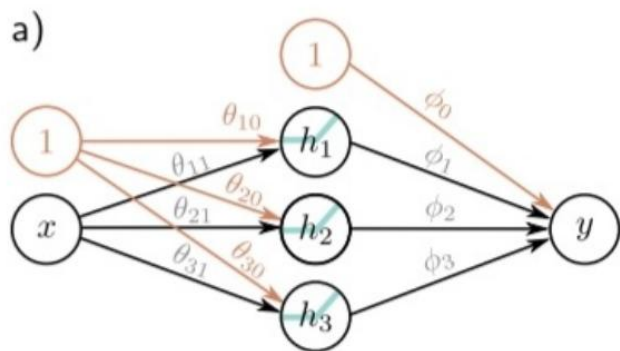
파라미터 = $\phi_0, \phi_1, \phi_2, \phi_3$ 와 $\theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}$ — 모두 10개.

필기

은닉층이 **하나뿐**이라 얇은(shallow) 신경망

신경망 그림으로 그리기 Depicting Networks

같은 계산을 회로도처럼 노드와 화살표로 표현합니다.



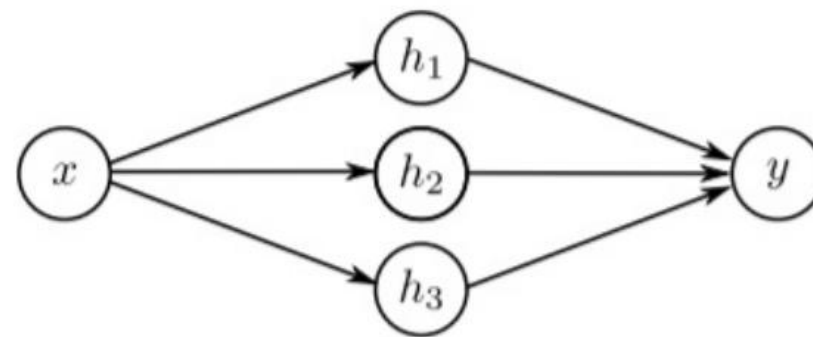
절편(1) · ReLU 포함

Figure 3.4a

필기

왜 1이 추가됨? $\theta_{10} + \theta_{11}x = \theta_{10} \times 1 + \theta_{11}x$ — 절편을 '1' 노드로 표현
 모든 화살표마다 파라미터가 하나씩 붙는다.
 [화살표 개수 = 파라미터 개수]

화살표 각각 하나의 파라미터가 붙는다.



간단 버전

Figure 3.4b

필기

(b)는 (a)의 **간단 버전(상수1 생략)**. Figure 3.3을 회로도처럼 그린 것

은닉 유닛 Hidden Units

$$y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

중간 계산값 h_1, h_2, h_3 를 은닉 유닛(은닉 뉴런)이라 부릅니다.

$$h_1 = a[\theta_{10} + \theta_{11}x], \quad h_2 = a[\theta_{20} + \theta_{21}x], \quad h_3 = a[\theta_{30} + \theta_{31}x]$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$



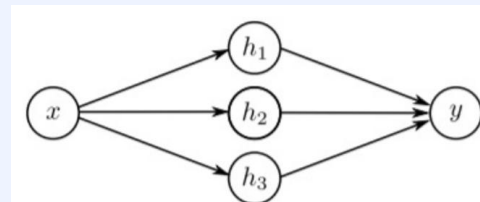
필기

Hidden layer = 은닉층.

그 안의 계산 단위 h_i 가 hidden unit(= 은닉 뉴런)입니다.

필기

뉴런 한 개는 $h_1 = \text{ReLU}(\theta_{10} + \theta_{11}x)$. 이런 뉴런 세 개가 모여 하나의 은닉층을 이룹니다.



파라미터의 역할 $y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$

식을 두 부분으로 나누면 θ 와 ϕ 의 역할이 분명해집니다.

θ · 모양 만들기

θ

직선을 만든다

은닉 유닛(뉴런)마다 $\theta_0 + \theta_1 x$ 라는 1차 함수(직선)를 만든다.
 θ 가 절편과 기울기를 정해 **직선의 모양**을 결정한다.

필기

각각의 $\theta_0 + \theta_1 x$ 는 직선

ϕ · 섞기

ϕ

직선을 섞는다

활성화를 거친 결과(활성화 함수화한 직선들)를 ϕ 로 가중합하고
 ϕ_0 로 전체 높이를 조절한다. ϕ 가 **어떻게 섞을지**를 결정한다.

핵심

θ 는 중간 직선들의 모양을, ϕ 는 그 직선들을 어떻게 섞을지를 결정

ReLU 활성화 함수 ReLU

$$y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

활성화 함수 $a[\cdot]$ 중 가장 널리 쓰이는 함수

$$a[z] = \text{ReLU}[z] = \max(0, z)$$

필기

$a[\cdot]$ 는 활성화 함수(activation function)를 뜻하는 기호

필기

ReLU : z (직선)의 결과값이 음수일때만 0으로 보내고, 양수는 건드리지 않는다.
이 간단한 비선형이 신경망의 핵심

$$a[z] = \text{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$

$$\text{ReLU}[z] = \max(0, z)$$

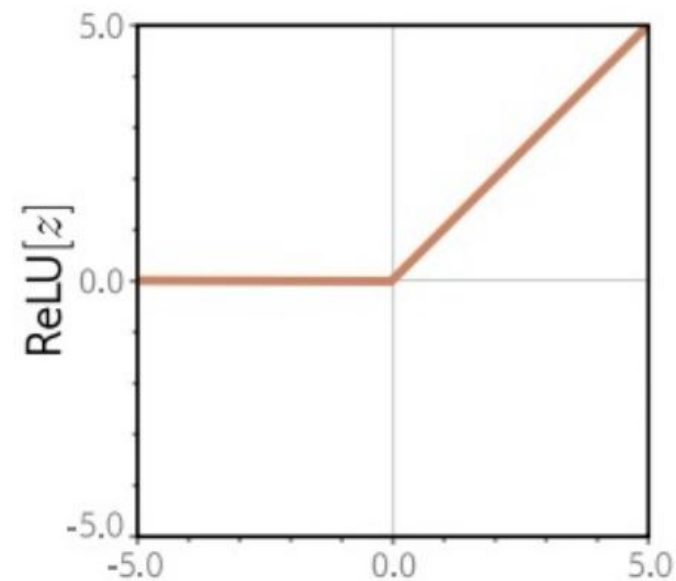
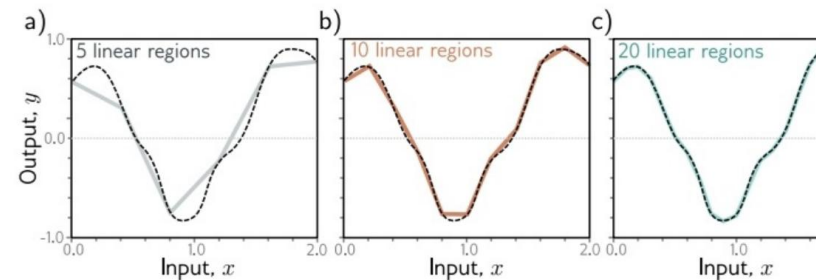


Figure 3.1

왜 비선형이 필요한가 Why Nonlinearity

활성화 함수(여기선 ReLU)가 없으면 신경망은 직선 하나로 무너집니다.



활성화가 없으면

선형 변환을 아무리 쌓아도 그냥 통과(identity)됩니다. linear $\rightarrow$ linear $\rightarrow \dots \rightarrow$ linear = 결국 **하나의 선형**.
표현력이 늘지 않습니다. (복잡한 곡선 함수 근사 불가)
그저 직선.평면.초평면 하나로 남게될 뿐.

ReLU가 있으면

음수를 꺾어 **비선형**을 도입 $\rightarrow$ 조각별 선형으로 어떤 곡선.곡면도 근사.

직관

$$y = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

ReLU에서 양수는 살리고 음수는 죽이는 이유

양수 = 특징이 존재함(예: '고양이'를 강하게 찾음 = 유용한 정보)

음수 = 정보 없음. 없는 정보를 죽이는 편이 효율적

[계산 ①] 세 직선 만들기 Step 1 · Three Lines

먼저 입력으로부터 세 개의 1차 함수를 계산합니다.

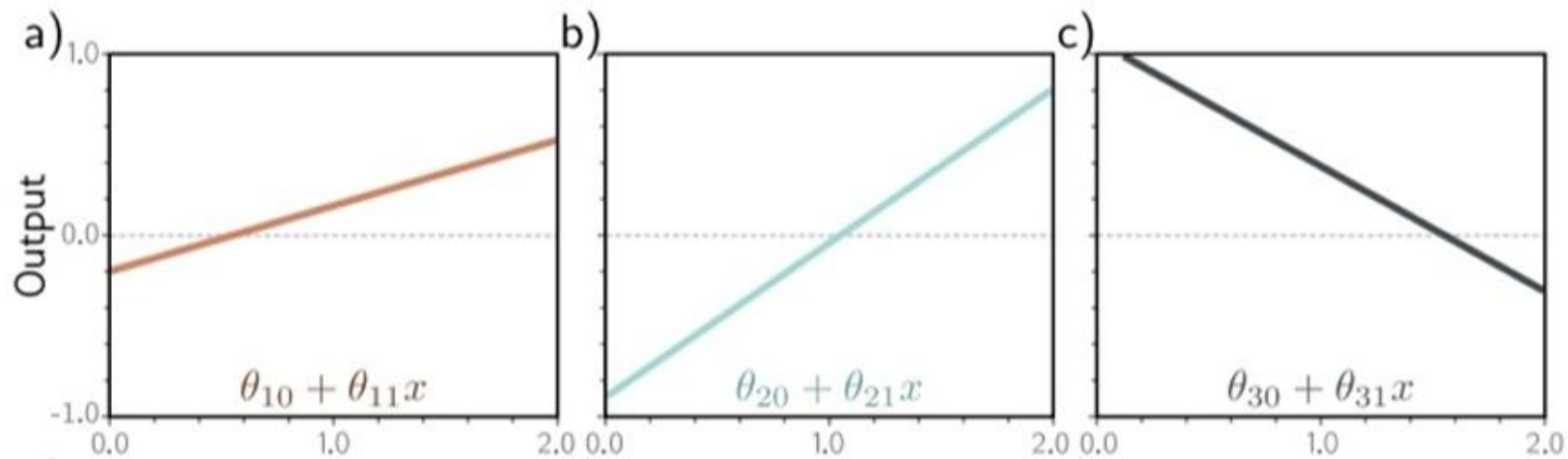


Figure 3.3 a-c

필기

각 은닉 유닛 속 $\theta_{\bullet 0} + \theta_{\bullet 1}x$ 는 1차 함수(직선). 아직은 그냥 평범한 직선들 !

[계산 ②] ReLU 통과 Step 2 · Apply ReLU

세 직선을 각각 ReLU에 통과시킵니다.

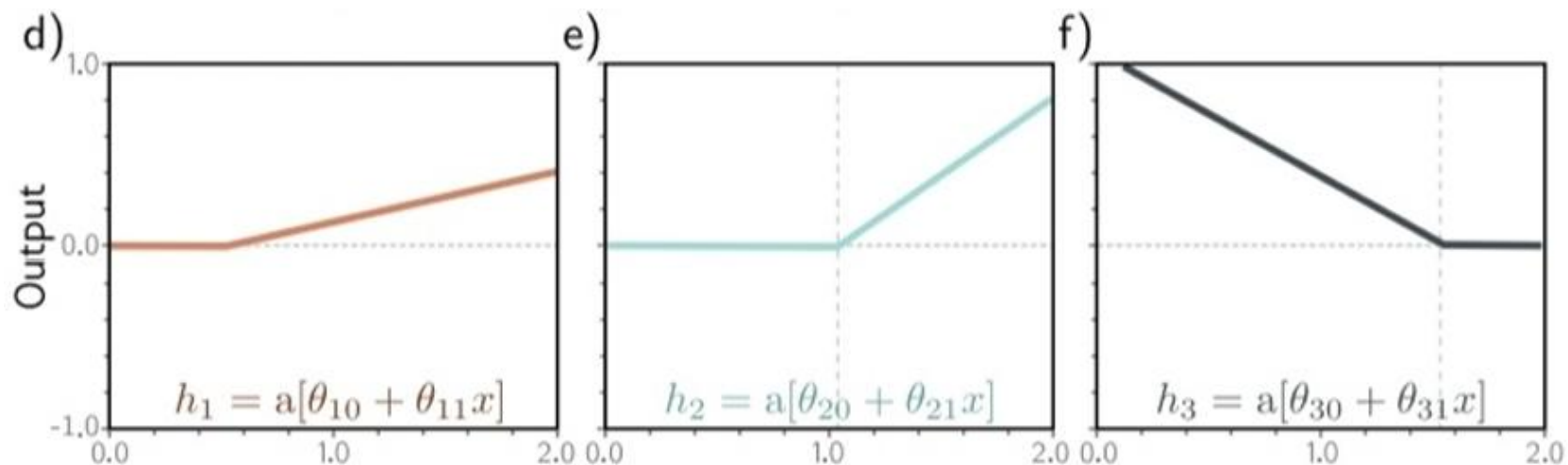


Figure 3.3 d-f

필기

직선들이 **ReLU를 만나다!!** 음수 부분이 0으로 잘려 꺾인 모양 h_1, h_2, h_3 가 됩니다.

[계산 ③] ϕ 로 가중 Step 3 · Weight by ϕ

꺾인 함수들에 각각 가중치 ϕ 를 곱합니다.

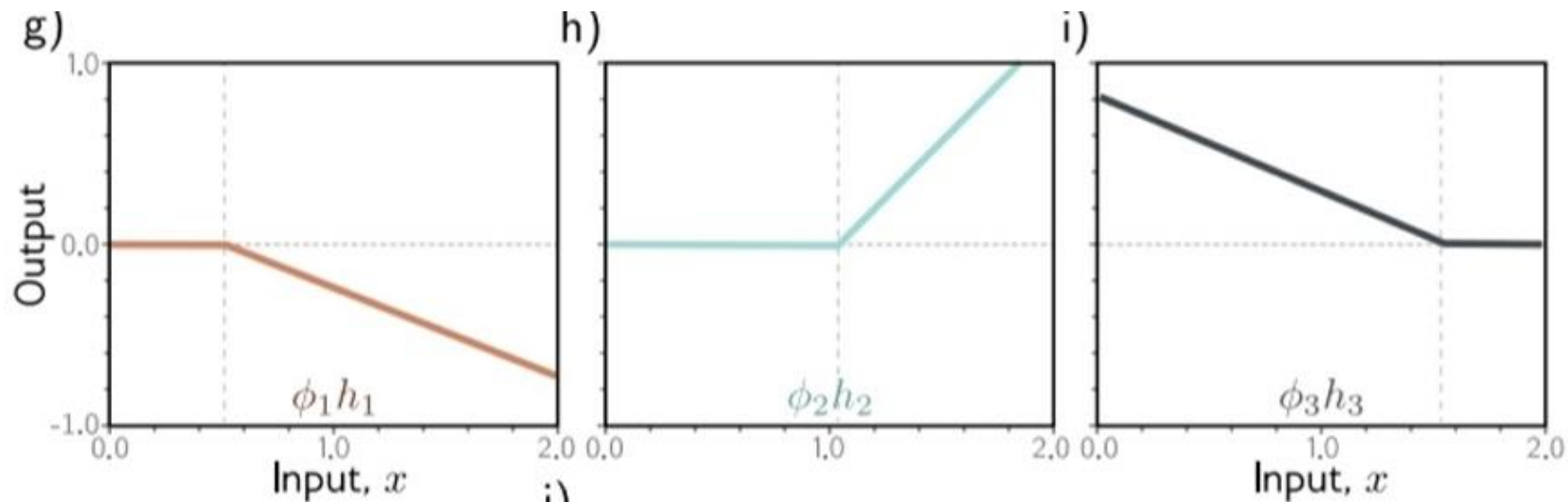


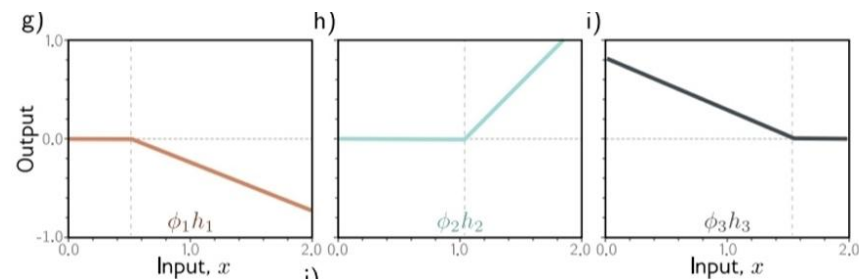
Figure 3.3 g-i

필기

각각에 ϕ 를 곱합니다. ϕ 의 부호·크기에 따라 기울기가 커지거나 뒤집힙니다.

[계산 ④] 합산 → 영역 Step 4 · Sum

가중된 함수들을 모두 더하면 하나의 조각별 선형 함수가 됩니다.



합산 결과

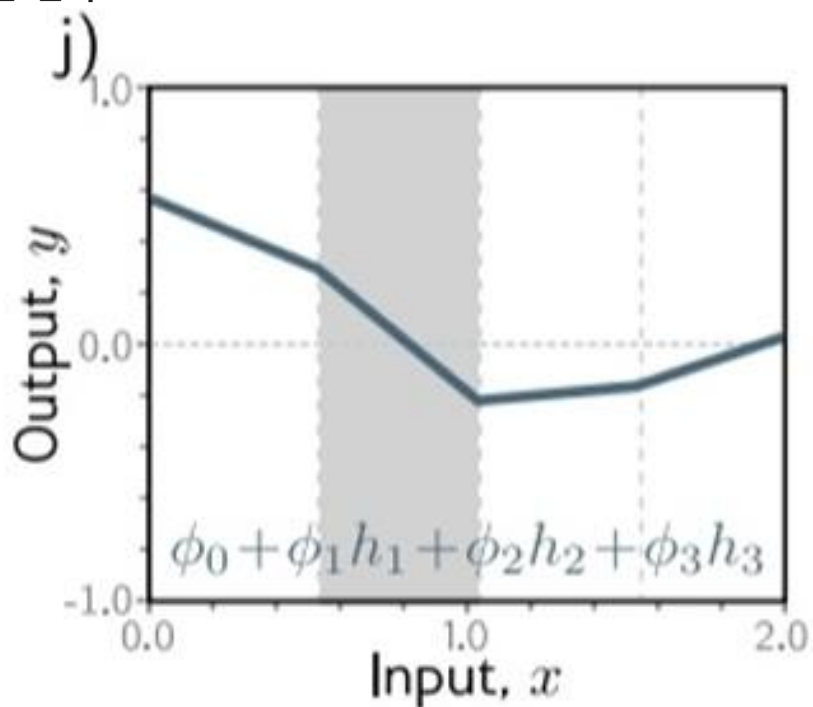


Figure 3.3 j

필기

joint : ReLU가 꺾이는 지점. 직선 개수만큼 꺾이며, 여기선 3번 꺾인다.

linear region : joint 사이 직선 구간.

입력이 1개면(1D) 최대 linear region = hidden unit + 1 = 꺾이는 지점 + 1

입력이 여러 개면 최대 linear region -> 뒤에 나옴

필기

“최대” linear region인 이유 — joint 위치가 겹칠 수 있고, 기울기가 우연히 같을 수도 있기 때문.

필기

activation pattern : 영역마다 켜진(활성) 은닉 유닛 조합이 다릅니다.

예) 특정한 x에서 두 번째 직선의 y값이 음수였고 나머지 양수였다면 이때 활성화된 은닉 유닛은 첫 번째와 세 번째 유닛 뿐이다.

함수의 가족 Family of Functions

같은 식, but 다른 파라미터 — 다양한 조각별 선형 함수가 만들어집니다.

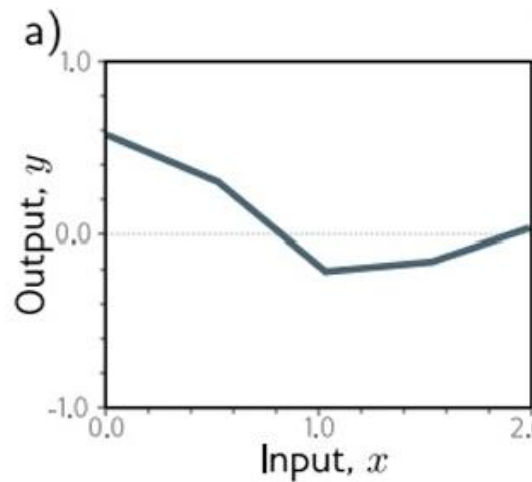


Figure 3.2a

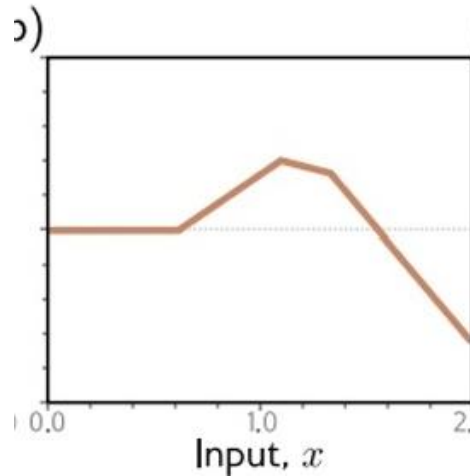


Figure 3.2b

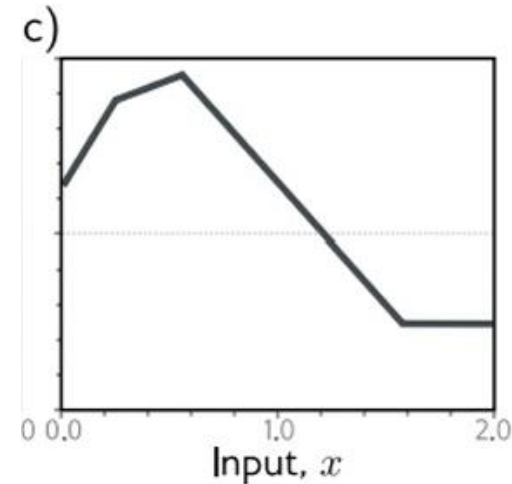


Figure 3.2c

필기

파라미터가 바뀌면 세 가지가 바뀝니다 — ① 꺾이는 위치(θ) ② 각 구간의 기울기(φ) ③ 전체 높이(φ_0).

일반화와 UAT(만능 근사) Generalization & UAT

은닉 유닛을 3개에서 D개로 확장 — 충분히 많으면 어떤 함수든 근사합니다.

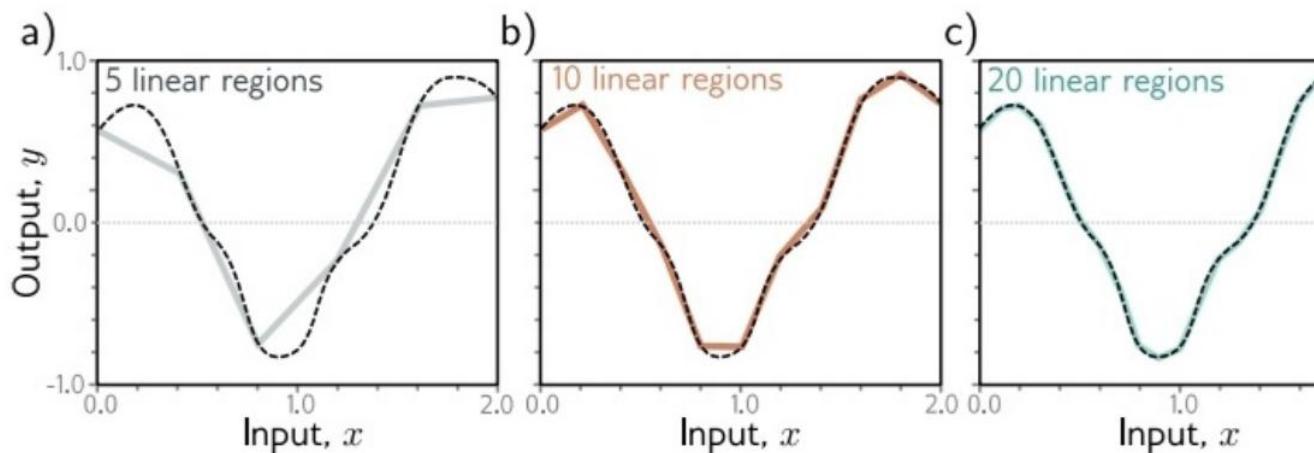


Figure 3.5 · 은닉 5 / 10 / 20개

$$h_d = a[\theta_{d0} + \theta_{d1}x] \quad y = \phi_0 + \sum_{d=1}^D \phi_d h_d$$

D + 1

은닉 D개일 때 최대 linear region 수(단, 입력이 1차원)

정리

Universal Approximation Theorem

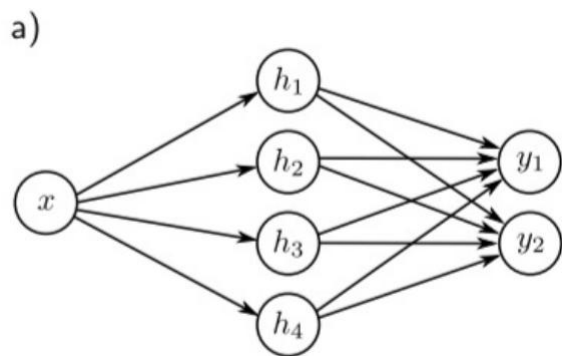
: 은닉 유닛이 충분히 많으면 임의의 연속함수를 원하는 만큼 정확히 근사

일반화

<은닉 유닛을 3개에서 D개로 일반화한 얇은 신경망

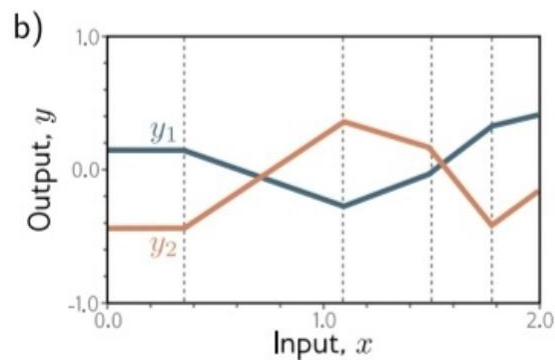
다중 출력 (y 多) Multivariate Outputs

출력이 여러 개여도 같은 원리 — 은닉 유닛을 공유(h1, h2, h3, h4 동일)



1입력 · 4은닉 · 2출력

Figure 3.6a



두 출력 y_1, y_2

Figure 3.6b

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

$$h_4 = a[\theta_{40} + \theta_{41}x]$$

$$y_1 = \phi_{10} + \phi_{11}h_1 + \phi_{12}h_2 + \phi_{13}h_3 + \phi_{14}h_4$$

$$y_2 = \phi_{20} + \phi_{21}h_1 + \phi_{22}h_2 + \phi_{23}h_3 + \phi_{24}h_4$$

필기

이제 입력·출력이 하나가 아니라 $x_1.., y_1..$ 여러 개입니다.
여기선 입력 1개, 출력 2개.

필기

y_1, y_2 는 은닉 유닛을 공유 → joint 위치는 동일, 기울기(ϕ)만 다릅니다.

다중 입력 (x 多) Multivariate Inputs

입력이 여러 개면 은닉 유닛은 직선이 아니라 평면/초평면이 됩니다.

2입력 신경망

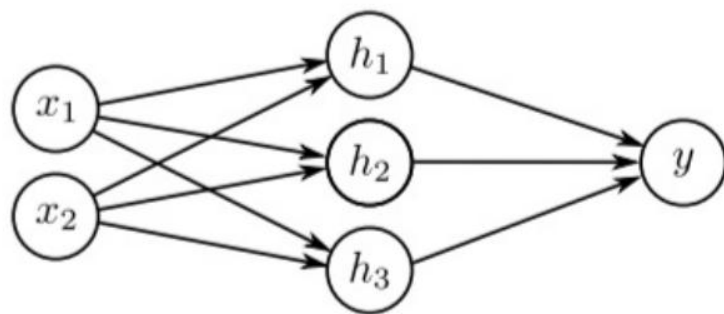


Figure 3.7 · 2입력 · 3은닉 · 1출력

필기

직선 – 입력이 1개일 때 선형식이 만드는 것

평면 – 입력이 2개일 때 선형식이 만드는 것

초평면 – 입력이 2개 이상일 때 선형식이 만드는 것
(고차원에서 직선/평면의 일반화)

$$h_1 = a[\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2]$$

$$h_2 = a[\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2]$$

$$h_3 = a[\theta_{30} + \theta_{31}x_1 + \theta_{32}x_2]$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

필기

이번엔 입력 2개·출력 1개. 은닉 유닛이 직선이 아니라 평면 $\theta_0 + \theta_1 x_1 + \theta_2 x_2$ 입니다. 예전 $h_1 = \text{ReLU}(\theta_{10} + \theta_{11}x_1)$ 가 입력 둘이면 $h_1 = \text{ReLU}(\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2)$ 로 확장됩니다.

[2D 처리] ① 평면 2D · Planes

두 입력의 선형결합은 기울어진 평면을 만듭니다.

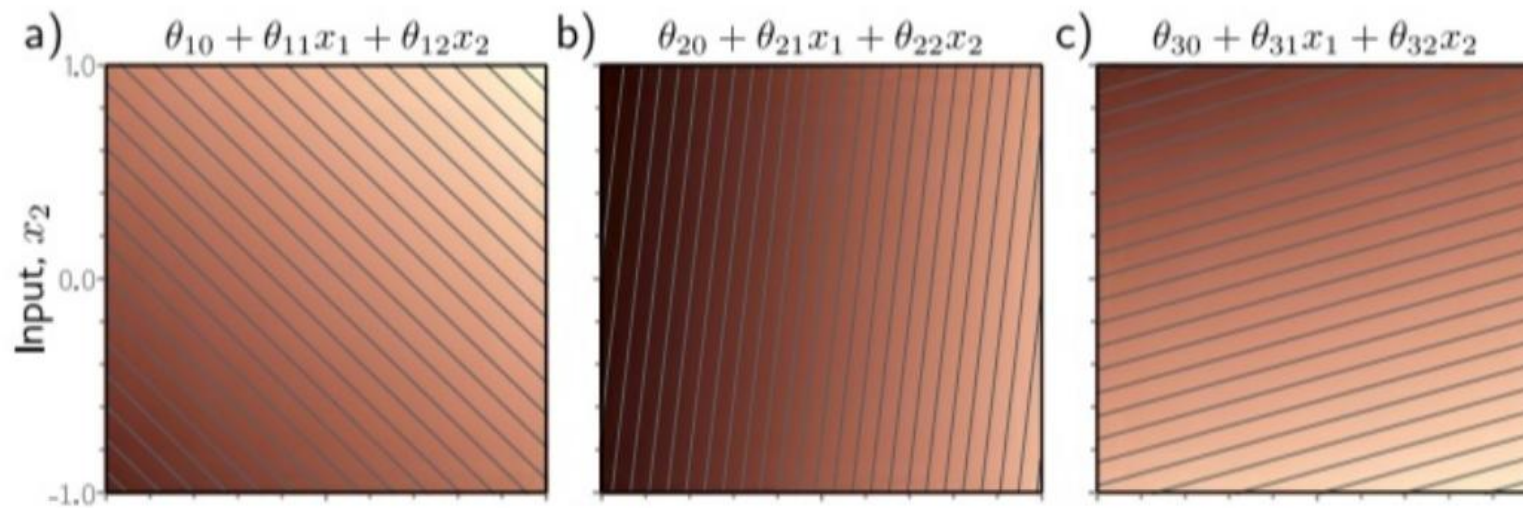


Figure 3.8 a-c

필기

각 은닉 유닛 = 두 입력의 선형결합 = 기울어진 **평면**. (θ_1, θ_2 의 값에 따라 기울어진다. 예) $z = \theta_0 + \theta_1x_1 + \theta_2x_2$ 에서 $\theta_1=1, \theta_2=0$ 이면 이 평면은 x_1 방향으로 기울어짐)
 밝은 부분 = 값이 큼, 어두운 부분 = 값이 작음.

[2D 처리] ② ReLU 적용 2D · ReLU

평면에 ReLU를 적용해 절반을 0으로 보냄.

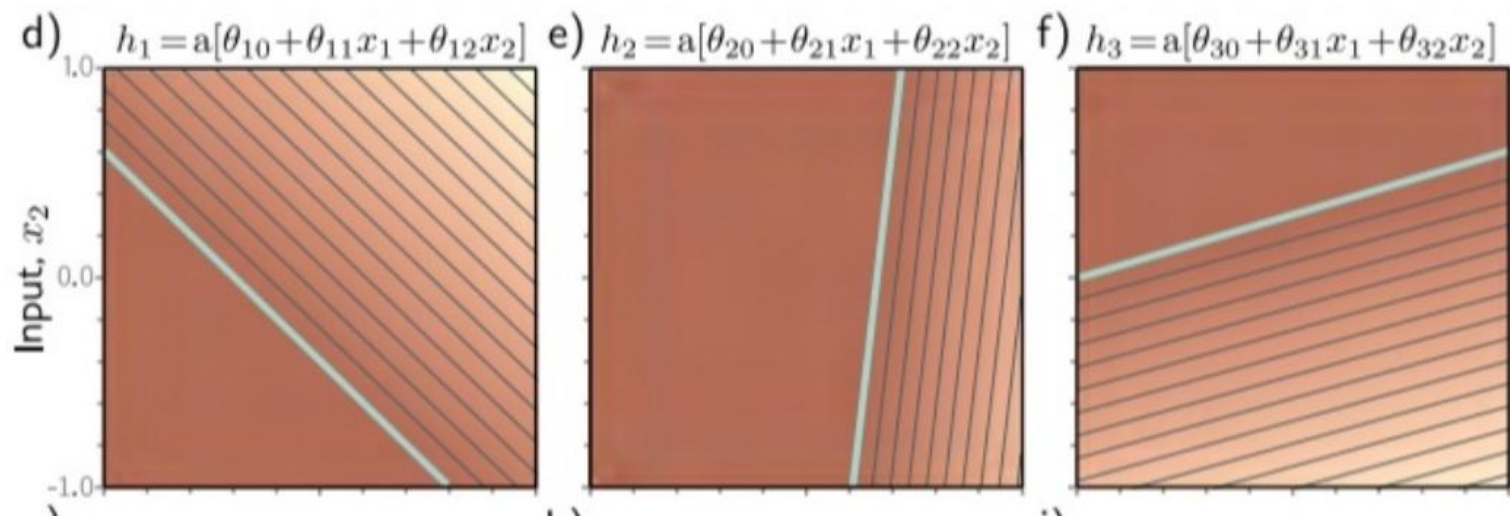


Figure 3.8 d-f

필기

ReLU 적용! 파란 선이 $z = 0$. 이 선을 기준으로 어두운 부분은 날아가고 **밝은 부분만 살아남습니다.**

[2D 처리] ③ 가중·합산 2D · Sum

잘린 평면들을 ϕ 로 가중합하면 하나의 곡면이 됩니다.

합산 결과

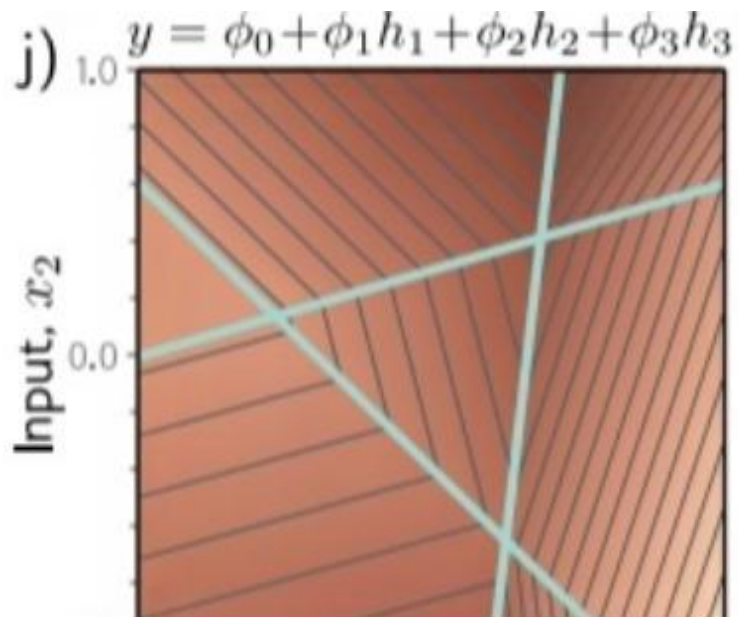
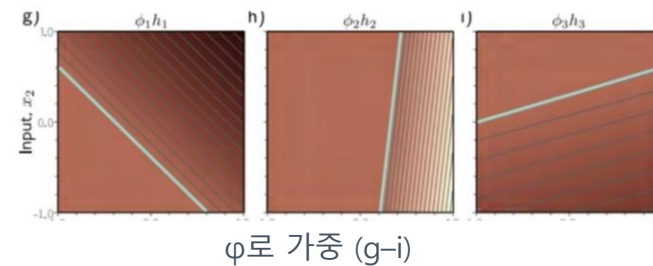


Figure 3.8 j



필기

가중치 ϕ : 기울기가 달라지거나(등고선 촘촘/띄엄) 부호가 뒤집힙니다.

필기

$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3 \rightarrow$ 하나의 곡면.

Like 종이접기를 접었다 편 자국처럼 평평한 면들이 이어진 모양이다!

일반식 General Case

입력 D_i 개 · 은닉 D 개 · 출력 D_o 개로 일반화한 식입니다.

3입력 · 3은닉 · 2출력

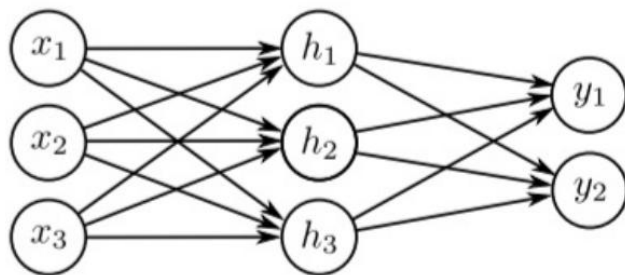


Figure 3.11

필기

$$3 \times 3 + 3 \times 2 = 15$$

$$1 \times 3 + 1 \times 2 = 5$$

파라미터 개수 : 15 + 5

$$h_d = a \left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i \right]$$

$$y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d$$

20

파라미터 (15+5)

필기

D_i = 입력 수, d = 은닉 유닛 번호.

activation pattern — 영역마다 켜진 은닉 유닛이 다름.

예) 한 영역에서 $h_2 = 0$ 이면 $y = \phi_0 + \phi_1 h_1 + \phi_3 h_3$.

입력 차원과 영역 Regions vs Dimensions

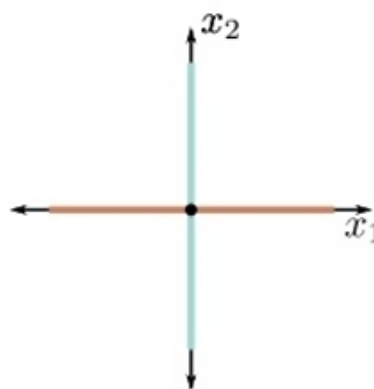
입력 차원이 커질수록 공간이 더 잘게 나뉩니다.

a)



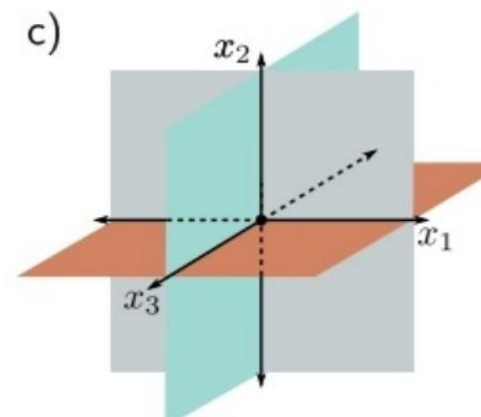
입력 1개 → 축 2개

b)



입력 2개 → 평면 4개

c)



입력 3개 → 공간 8개

필기

입력 차원이 커질수록 초평면(hyperplane)이 공간을 잘게 나뉘어 표현력이 매우 빠르게 증가합니다.

$$2^{D_i}$$

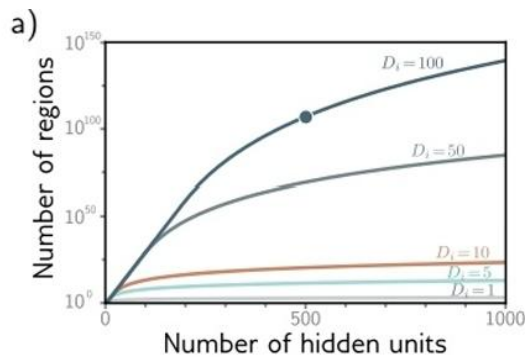
필기

입력 1→2, 2→4, 3→8 ... 일반적으로 2^{D_i} 만큼의 공간이 생긴다. 평면이 기울어져 있어 더 많이 생길 수도 있다. (D_i : 입력 개수)

Linear region의 폭발 Explosion of Capacity

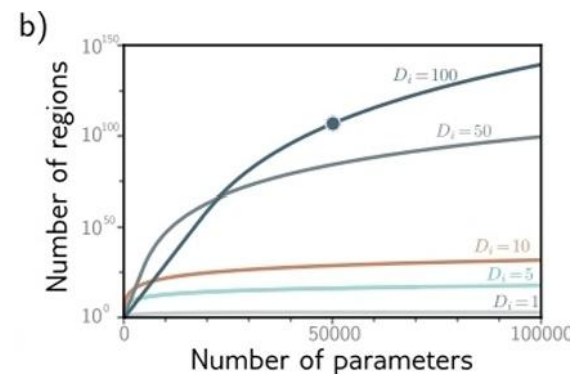
은닉 유닛이 늘면 linear region이 폭발적으로 증가합니다.

파라미터 : 은닉 유닛이 증가할수록 증가



은닉 유닛 수에 따른 region

Figure 3.9a



파라미터 수에 따른 region

Figure 3.9b

$$N_{\text{regions}} \leq \sum_{j=0}^{D_i} \binom{D}{j}$$

최대 linear region 수 (D_i =입력 차원, D =은닉 유닛 수)

필기

꺾이는 지점 = **Joint** (= 은닉 unit 개수)

두 joint 사이 = **Linear region**.

입력이 **100차원**이면 은닉을 조금만 늘려도 region이 폭발적으로 증가 → 표현력 ↑. **입력 1차원($D_i=1$)이면 최대 linear region = 은닉 유닛 수(D) + 1.**

활성화 함수의 종류 Activation Functions

ReLU 외에도 다양한 활성화 함수가 있습니다.

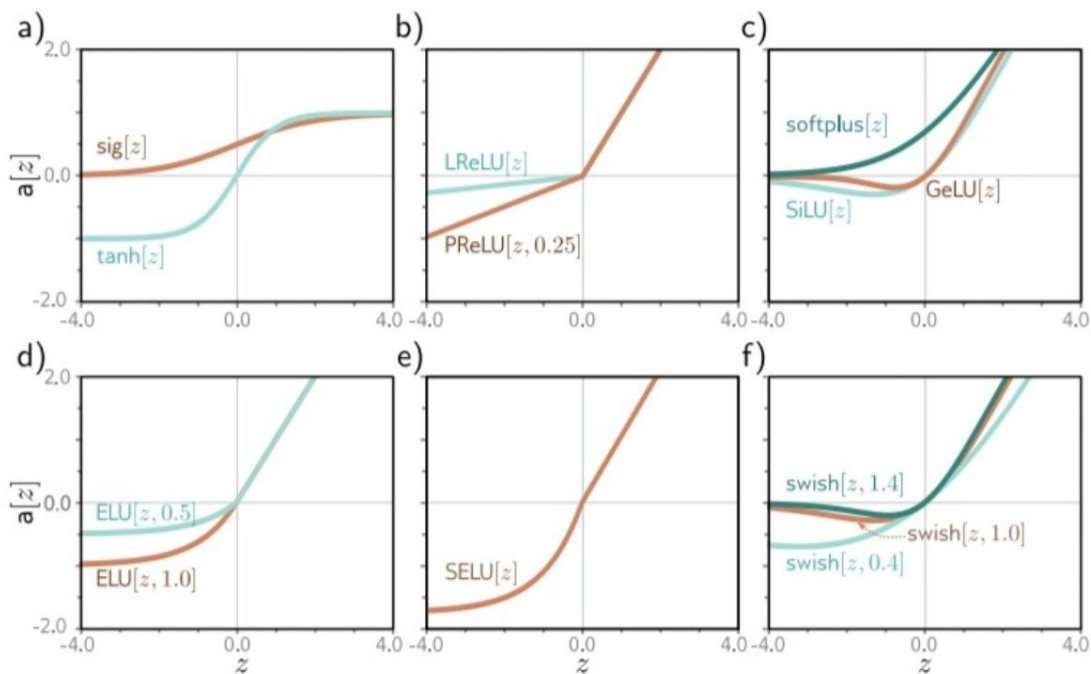


Figure 3.13

주요 함수 Cheatsheet

- a) Sigmoid $\sigma(x)=1/(1+e^{-x})$, 0~1, 확률 출력
- Tanh(하이퍼볼릭 탄젠트/텐 에이치) $-1\sim 1$, 평균 0, sigmoid보다 학습 잘됨(가중치가 한 방향으로만 업데이트 되는 문제 감소)
- B) Leaky ReLU $x < 0 \rightarrow 0.01x$
- PReLU 0.01 대신 α 를 학습(기울기를 0.01로 놓지 않고 α 를 학습)
- c) Softplus ReLU를 부드럽게(미분 가능)
- GeLU 부드럽게 통과, 작은 음수 일부 반영
- SiLU $x \cdot \sigma(x)$
- d) ELU 음수에서 지수함수, 평균 0 근처
- e) SELU ELU 개선, 깊은 신경망 분포 안정
- f) Swish $x \cdot \sigma(\beta x)$, ReLU보다 약간 좋은 성능

순전파와 역전파 Forward & Backprop

계산은 순전파로, 학습(가중치 수정)은 역전파로 합니다.

순전파 · Forward

입력 → 은닉 → 출력 방향으로 값을 **계산**만 합니다 (forward propagation).

역전파 · Backprop

출력 → 은닉 → 입력 방향으로 **오차**를 전달해 가중치를 수정합니다 = **학습**.

Input → Linear → ReLU → Hidden → Linear → Output

$X(\text{input}) \rightarrow \theta_0 + \theta_1 x$ (linear) $\rightarrow \text{ReLU}(\theta_0 + \theta_1 x)$ (ReLU, Hidden) $\rightarrow \varphi_0 + \varphi_1 \text{ReLU}(\theta_0 + \theta_1 x)$ (linear, output)

$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w}$$

필기

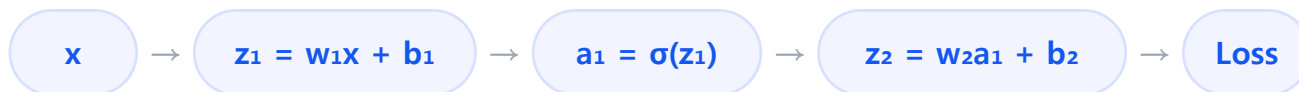
학습 = 가중치를 조금씩 수정해 더 잘 맞게 만드는 것. 역전파가 그 수정량을 정합니다. 갱신식 $w_{\text{new}} = w - \eta \cdot \partial L / \partial w$ (η = 학습률, learning rate).

기울기 소실 Vanishing Gradients

미분이 0에 가까우면 역전파 시 가중치(w)가 거의 수정되지 않음(sigmoid, tanh의 입력값이 조금만 커지거나 작아지면 기울기 소실 문제 발생)

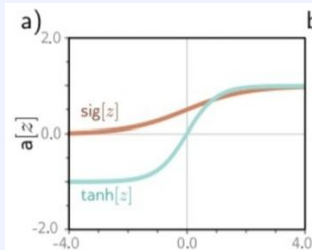
$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w}$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$



기울기 소실이란?

기울기가 0이면 w_{new} 가 거의 안 변함
→ 틀렸는데도 수정 불가. 이것이 기울기 소실.



필기

연쇄법칙 : $\partial L / \partial w_1 = (\partial L / \partial z_2)(\partial z_2 / \partial z_1)(\partial z_1 / \partial w_1)$. 활성화($a[\cdot]$)의 미분이 0에 가까우면 손실(Loss)의 미분도 0이 됩니다.

기울기 소실 해결을 위한 ReLU

Sigmoid와 tanh는 절대값이 큰 입력에서 대부분 기울기 0 → 그래서 큰 입력에서도 기울기가 유지되는 **ReLU**를 선호합니다. (정보량이 높은 양수 부분 보존 가능)

But, ReLU의 또다른 문제점

Dying ReLU — ReLU는 양수면 기울기 1($1 \times 1 \times 1 = 1$)이라 잘 전달되지만, 만일 모든 입력 데이터에 대해 어떤 뉴런이 항상 음수만 출력한다면 그 뉴런은 모든 데이터에 대해 기울기가 0이 되어 해당 뉴런의 모든 가중치는 거의 수정되지 못함.

Dying ReLU

Dying ReLU에 대한 자세한 설명 ; 뉴런과 연결된 모든 파라미터가 갱신되지 않는 이유

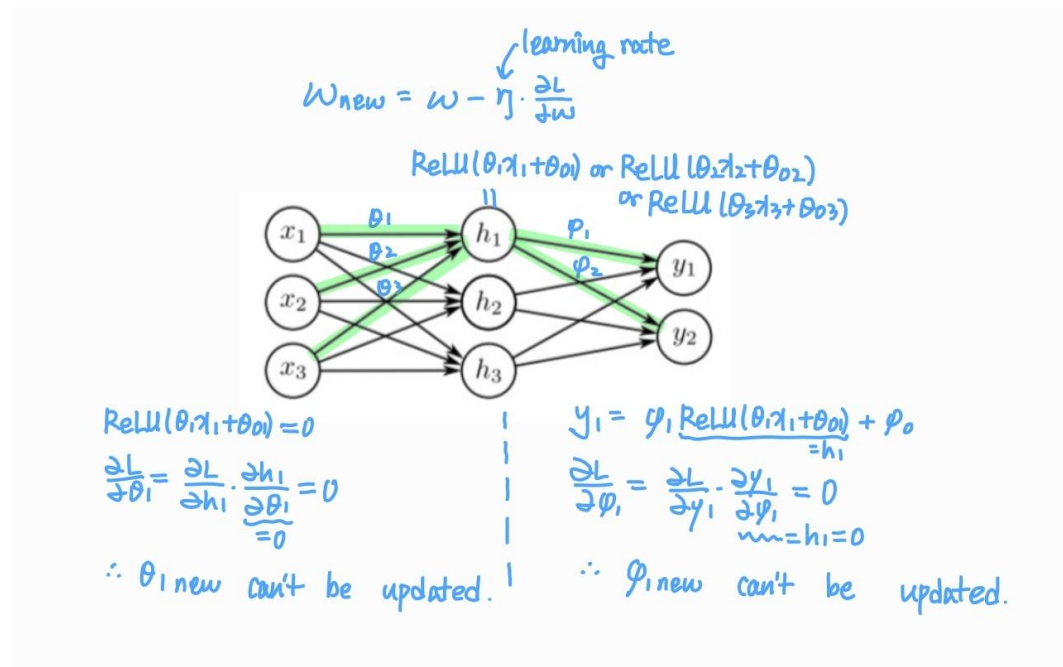


Figure 3.11 + 제 필기

$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w}$$

- 따라서 하나의 은닉 뉴런의 입력값이 항상 음수면, 0을 출력하여 해당 뉴런은 모든 가중치에 대해 기울기가 0이 된다. 그러면 그 뉴런과 연결된 가중치는 업데이트 될 수 없다.
- 초록 형광펜 친 부분은 업데이트 되지 않는 가중치들
- 해당 뉴런과 연결된 모든 파라미터들은 다 갱신될 수 없다.
- Dying ReLU 문제 해결을 위해 음수에도 기울기를 제공하는 Leaky ReLU, ELU같은 업그레이드 된 형태 사용하기도 함

결론: 왜 ReLU인가

Why ReLU

여러 활성화 함수가 있고, Dying ReLU 문제도 있지만, 여전히 ReLU를 씁니다.

단순함 Simple

음수 $\rightarrow 0$, 양수 그대로.
가장 이해하기 쉬운 비선형.

분석 용이 Analyzable

조각별 선형 $\rightarrow$ linear region으로 표현력
을 깔끔하게 분석 가능

기울기 전달 Gradient

양수 기울기 1 $\rightarrow$ 기울기 소실이 덜해 학습이 잘 됩니다.

결론

다양한 활성화 함수가 있지만 **ReLU**가 가장 설명하기 쉽고 linear region 분석이 편하기 때문에, 이 책에서는 앞으로도 계속 ReLU를 사용합니다.

Dying ReLU로 인한 비용이 기울기 소실로 인한 비용보다 훨씬 적고, 발생 확률도 드물다. (직선이 모든 입력값에 대해 음수를 내놓을 확률은 생각보다 적음)

3장 요약 Chapter 3 Summary

얇은 신경망의 큰 그림 — 한 장으로 정리합니다.

ReLU

비선형의 핵심

$D + 1$

1D) 최대 linear region

$\theta \cdot \varphi$

모양 · 섞기

순전파 / 역전파

계산 / 학습

- 구조: 입력층 $\rightarrow$ 은닉층 $\rightarrow$ 출력층. 은닉층이 하나라 '얇은' 신경망이다.
- 함수 생성: 직선 $\rightarrow$ ReLU $\rightarrow \varphi$ 가중합 $\rightarrow$ 조각별 선형 함수. joint와 linear region으로 표현력을 낸다.
- 확장: 다중 입력·출력, 고차원에서 region이 2^{D_i} 꼴로 폭발 $\rightarrow$ 강력한 표현력(UAT).
- 학습: 역전파로 가중치를 수정하며, 기울기 소실을 피하기 위해 ReLU를 선호한다.

전체 정리 Putting It Together

2장과 3장을 하나의 흐름으로 잇습니다.



CHAPTER 2 · 학습의 틀

- 모델 = 함수 $y = f[x, \phi]$
- Loss로 오차를 수치화 (오차제곱합)
- 경사 하강법으로 최소화
- 과적합을 피해 일반화 확인
- 직선 하나의 한계를 인식

CHAPTER 3 · 표현력

- ReLU로 비선형 도입
- 직선→ReLU→가중합 = 조각별 선형 함수
- 다중 입출력 · 고차원 처리
- linear region이 폭발적으로 증가
- 역전파로 학습

UNDERSTANDING DEEP LEARNING · CH. 2-3

감사합니다

Q&A